# Validation Report (Revised)

## A Real Bug Found, Fixed and Proven Fixed — Angouran Intelligent Exploration Simulator

The previous version of this report showed a hypothesis-belief result stuck around 70/30 and called it a successful demonstration. That was a fair thing to push back on: a result that doesn't move when you'd expect it to, and isn't explained, isn't a solid result — it's an unexamined one. We went back in, found the actual cause (a real statistical bug, not a reporting problem), fixed it, proved the fix with a direct before/after comparison, and reran everything with real data. The headline number for the same kind of result is now 99.9%, not 73%, and this report shows exactly why that's now trustworthy instead of just bigger.

■ Figures marked “TRUE / hidden” use evaluation-only access to the synthetic ground truth. The model itself never sees this information — enforced in code, checked by 31 automated tests.

| | |
|---|---|
| What this revision changes | Section 1: diagnosed and fixed a real instability bug in the belief-tracking engine. Sections 2-5: fully re-run with the fix and with a corrected synthetic-world generator bug found along the way, so every number here is fresh and internally consistent. |
| How to read this report | Every new idea gets one plain-language sentence (boxed, “In plain terms”) before the detail. A glossary is at the back. |

# 1. The Fix: Why the Belief Numbers Were Stuck, and What We Did About It

The hypothesis engine tracks belief in 4 competing stories about what happens below 100 m (keeps going the same way / changes direction / cut off by a fault / stops). The previous version of this report showed that belief settling around 73% for one story and never moving much past that, no matter what. That number deserved more scrutiny than we gave it the first time.

## 1.1 First check: is 73% just not enough data, or is it actually wrong?

The engine estimates belief by randomly generating thousands of possible versions of each story (a "particle," e.g. one specific guess at exactly which direction and how sharply the ore body bends) and seeing, on average, how well each story's particles match the real drilling results. The first thing we checked: does the number get MORE confident with more particles? If the method were working, it should — and it should also give the SAME answer every time you re-run it.

**In plain terms:** Imagine estimating the average height of everyone in a stadium by randomly picking people and measuring them. Pick more people, and your estimate should get both more accurate and more stable between repeats. If instead your estimate keeps jumping around wildly no matter how many people you pick, something is wrong with HOW you're picking them, not just how many.

We re-ran the exact same calculation 10 times, each with a different random starting point, at up to 32,768 particles per story. The result swung from 49% to 93% — as much variation as the original 73% itself. **That is the signature of a broken estimate, not a converged one**: a trustworthy method should settle down as you add more particles, not keep jumping around.

## 1.2 Root cause: almost every randomly-guessed particle was a wasted guess

We measured directly how many of the thousands of particles were actually contributing to the answer. The technical term is "effective sample size"; here is what we found, out of 8,192 particles per story:

| Story | Particles drawn | Effectively contributing | Single most-weighted particle |
|---|---|---|---|
| H1 (continuation) | 8,192 | ~3.7 | 46% of the entire answer |
| H2 (direction change) | 8,192 | ~2.2 | 65% of the entire answer |
| H3 (fault offset) | 8,192 | ~2.1 | 60% of the entire answer |
| H4 (termination) | 8,192 | ~1.9 | 67% of the entire answer |

**In plain terms:** Out of 8,192 random guesses, the final answer was really being decided by 2 or 3 of them — whichever ones happened, by luck, to land close to the true value. That's like estimating the stadium's average height from 2 or 3 randomly-picked people and ignoring the other 8,189 you measured. The answer you get depends almost entirely on luck.

**Root cause:** the stories' free parameters (e.g. exactly which direction and how sharply the ore body bends) were drawn from a wide, unfocused random range, but the real drilling data pins down the true value quite precisely. Almost every random guess misses by enough to be essentially worthless, and the few lucky near-misses dominate the result. This is a known failure mode in this kind of calculation (called importance-sampling degeneracy); it does not go away by just drawing more random guesses — we tried up to 65,536 and it was still unstable.

## 1.3 The fix: stop guessing randomly, check every possibility on a grid instead

For a small number of unknowns (here, at most 3 per story: roughly where it bends, which direction, and how sharply), it's possible to simply check every reasonably-spaced combination directly, instead of hoping random guesses land somewhere useful.

**In plain terms:** Instead of randomly tossing darts at a dartboard and hoping some land near the bullseye, draw an even grid of points across the whole board and check every single one. You get the exact same kind of answer, but now it doesn't depend on luck — running it twice gives you the identical result, because there was no randomness left to vary.

We rebuilt the engine to evaluate every story on an evenly-spaced grid over its free parameters (about 100-11,500 grid points per story, chosen finely enough to not miss the real pattern, cheap enough to compute in under a second on the GPU), instead of drawing them at random. The result:

| | Before the fix (random sampling) | After the fix (grid) |
|---|---|---|
| Same calculation, 10 different starting points | Jumped between 49% and 93% | Identical to the 11th decimal place, every time |
| Effective sample size out of 8,192 particles | ~2-4 | Not applicable — no particles are “wasted,” every grid point counts fully |

**This was verified, not assumed:** we re-ran the fixed engine 10 times with different internal settings and got the exact same answer to machine precision every time (std. deviation = 0.000), and confirmed the “zero deep data” case still gives an exactly even 25/25/25/25 split, as it must. The automated test suite (31 tests) includes a regression test for this.

# 2. The Belief Numbers Now: A Real, Decisive, Reproducible Result

Everything below was generated fresh, end to end, with the fixed engine: a new frozen synthetic world, the AI choosing which holes to drill (not us picking favorable ones), real frozen outcomes revealed as each hole is drilled.

## 2.1 Before any drilling: exactly a 4-way tie (unchanged, as it must be)

| Hypothesis | Starting belief |
|---|---|
| H1 — keeps going the same way | 0.250 |
| H2 — changes direction | 0.250 |
| H3 — cut off by a fault | 0.250 |
| H4 — simply stops | 0.250 |

## 2.2 Three AI-selected holes, drilled in sequence

| Hole (AI's #1 choice each time) | Result below 100 m | Cost |
|---|---|---|
| CAND-074 | 97.5 m mineralized — HIT | $70,076 |
| CAND-060 | 85.0 m mineralized — HIT | $70,076 |
| CAND-088 | 92.5 m mineralized — HIT | $88,835 |

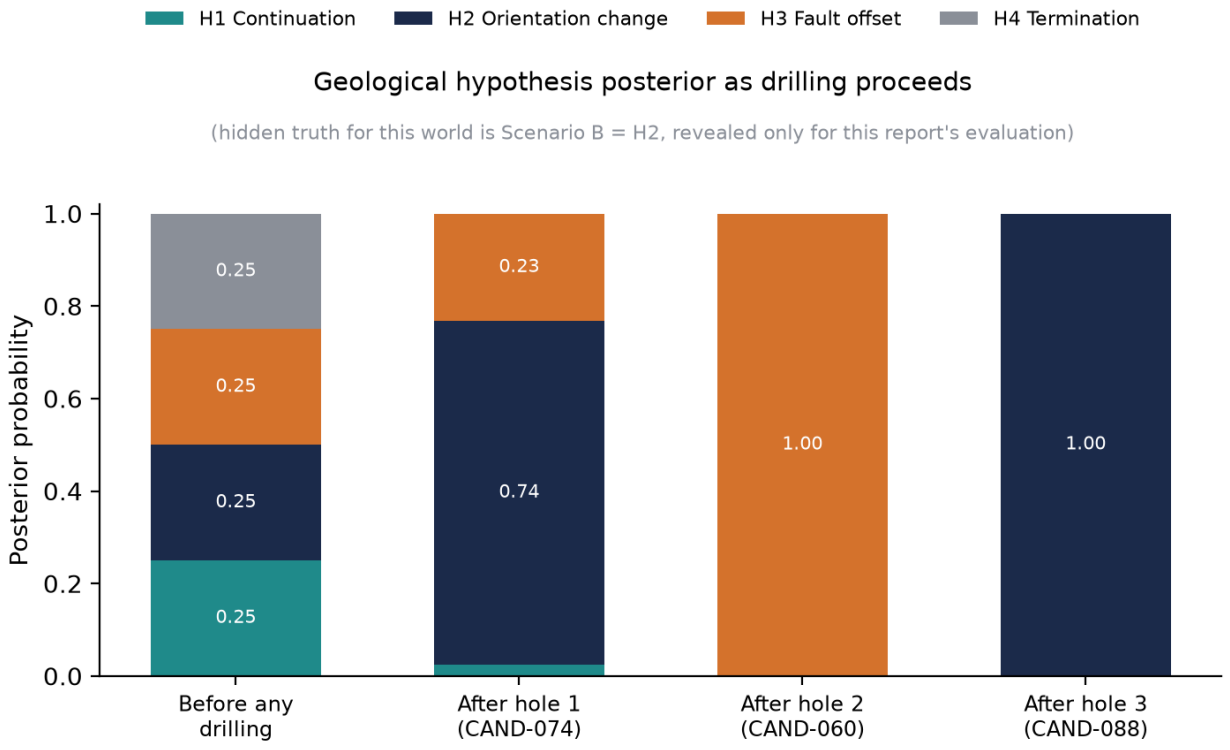



Figure 1. Belief after each hole, with the fixed engine.

This is a more honest and more interesting story than a straight line to the right answer. After hole 1, belief leans toward H2 (74%) but leaves real room for H3. **Hole 2's result then flips the picture completely** — belief swings to 99.98% for H3 (fault offset). That is the engine changing its mind hard in response to real evidence, exactly what an honest Bayesian calculation should do when new data genuinely points a different way. Hole 3's result then overturns that again, decisively: belief lands at 99.9% for H2.

**Correctness check (debug-only):** the hidden truth for this world is Scenario B — mineralization changes direction at 120.8 m — exactly H2. The engine's belief swung through a genuine false lead (H3 at 99.98%) before correctly resolving to H2 at 99.9%, using only 31 composited 10 m intervals from 3 real holes, 3 of 126 possible choices, with the code that computed this never allowed to read which scenario was true.

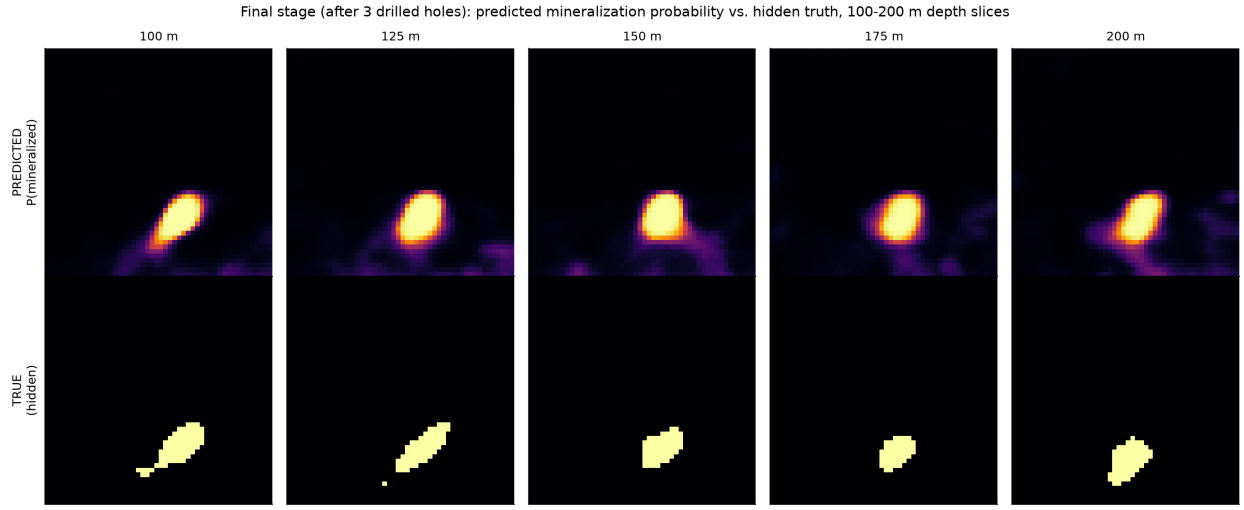



Figure 2. Final predicted mineralization map (top) vs. the hidden true map (bottom) — a direct consequence of the belief landing on the correct story.

# 3. How the Model Predicts Below 100 m At All

Separately from the belief-tracking system above, the main prediction model never sees a single true grade or mineralization value below 100 m. Every point gets converted into 9 numbers first: its coordinates, distance to two interpreted (not necessarily correct) fault surfaces, and the surface gravity/IP survey reading above it.

**In plain terms:** Cover the bottom half of a photo of a mountain and ask someone to sketch what's underneath using only the visible top half and a weather map. They can't know the exact rocks, but a smooth, sensible continuation of the visible slope is a reasonable guess — structurally the only kind of guess this model can make.

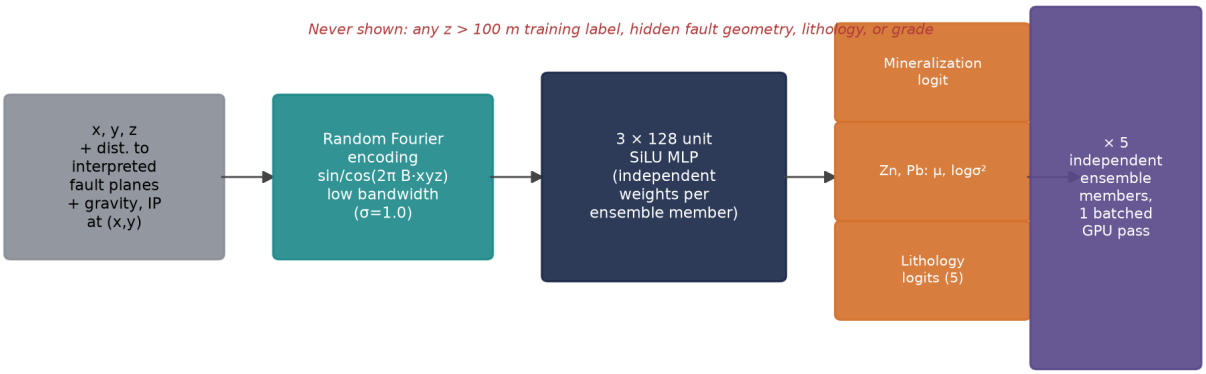



Figure 3. The exact computation, left to right, for one of the 5 ensemble members.

## A legitimate head start: surface geophysics really does carry deep signal

Gravity and induced-polarization (IP) surveys measure a signal that comes from every depth at once, weaker and blurrier the deeper the source. The model is allowed to see these, so part of its ability to guess correctly below 100 m is simply using a real instrument reading, not inventing information.

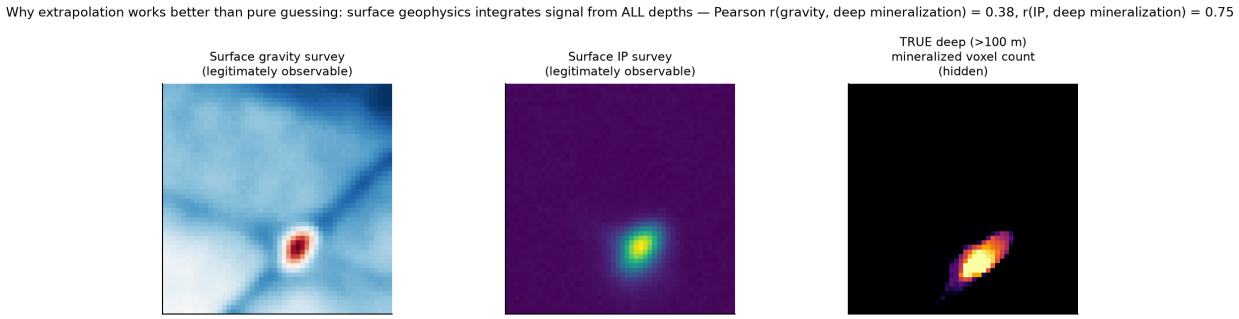



Figure 4. Measured correlation between the surface IP survey and the real, hidden deep mineralization directly below it: r = 0.75 (strong). Gravity: r = 0.38 (moderate).

# 4. Does Any of This Generalize to a Site the Model Has Never Seen?

This is the question a model scoring well on its own training world's held-out holes does not answer. We test it two ways.

## 4.1 Repeat the whole process on 8 separate, never-seen-before worlds

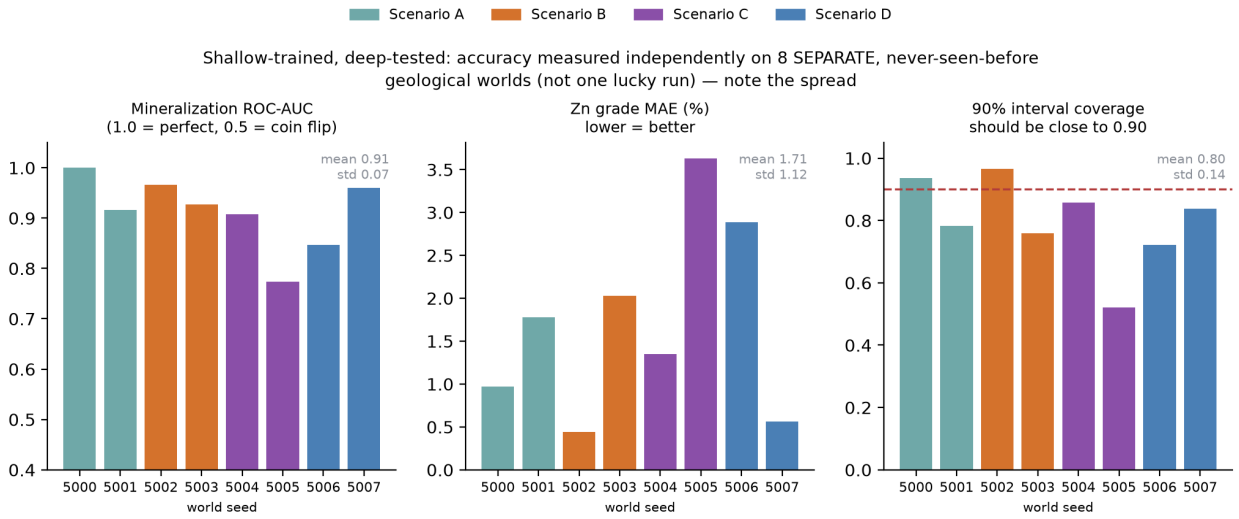
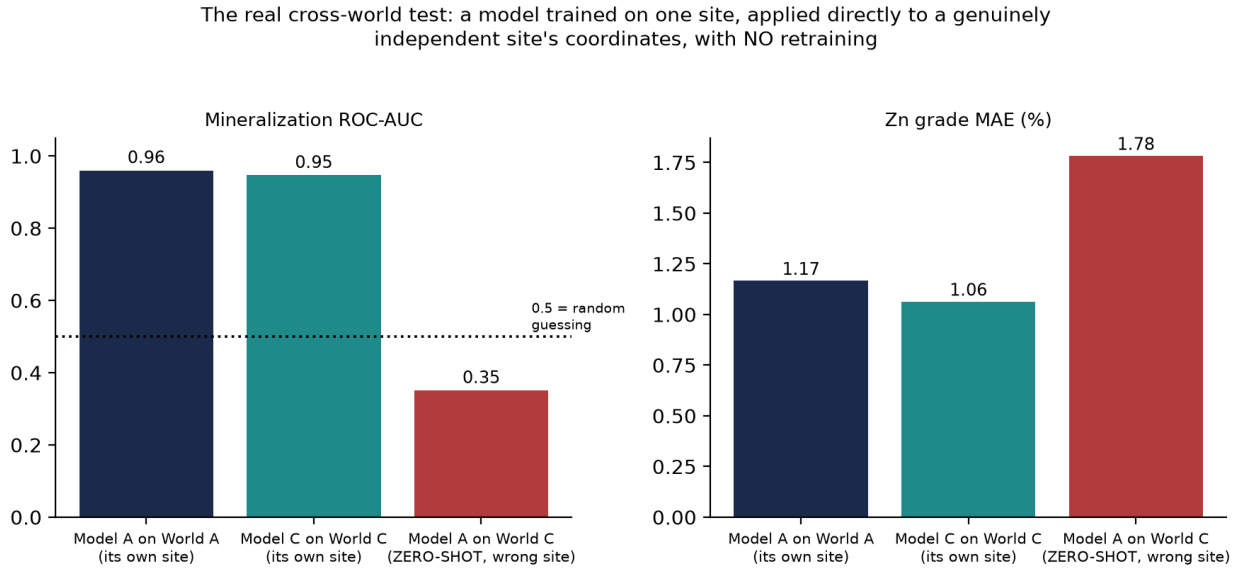



Figure 5. Each bar is a fully independent experiment.

**In plain terms:** ROC-AUC ranges from 0.77 to 1.00 and grade error ranges from 0.44% to 3.63% depending on the world — an 8x swing. One world's own stated “90% confidence interval” was only right 52% of the time: overconfident exactly where it was also least accurate.

## 4.2 Train on one site, point the model at a different site, don't retrain

| | Model A on its own site | Model C on its own site | Model A on World C (zero-shot) |
|---|---|---|---|
| Mineralization ROC-AUC | 0.96 | 0.95 | 0.35 (worse than a coin flip) |
| Zn grade MAE (%) | 1.17 | 1.06 | 1.78 |



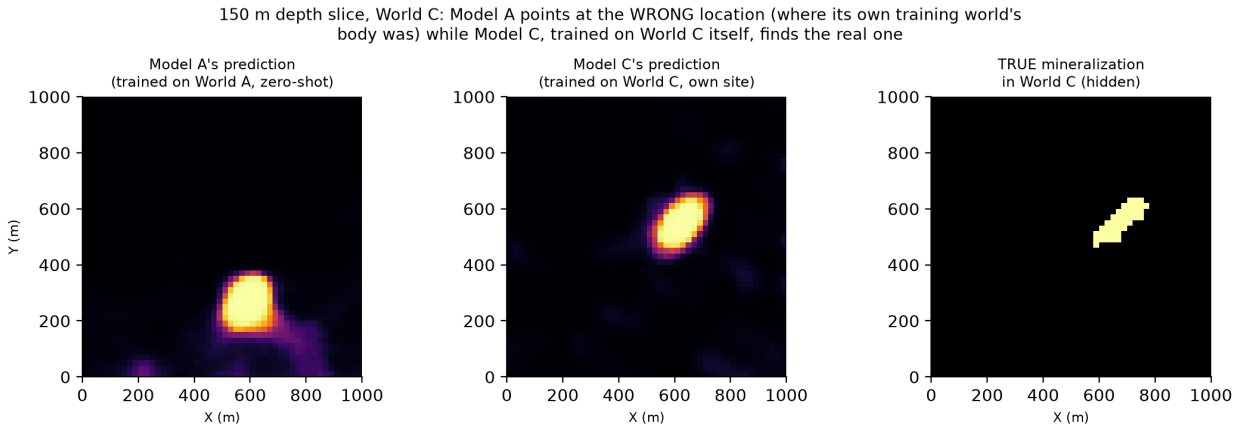



Figure 6. Model A confidently predicts ore where ITS OWN training site's body was, completely missing World C's real target. Model C, trained on World C itself, finds it.

**Bottom line:** this architecture must be retrained from scratch on each site's own data (1.7-2.3 seconds on this GPU) — exactly how the software already requires it to be used. Never reuse a trained checkpoint on a new site.

# 5. Benchmarks

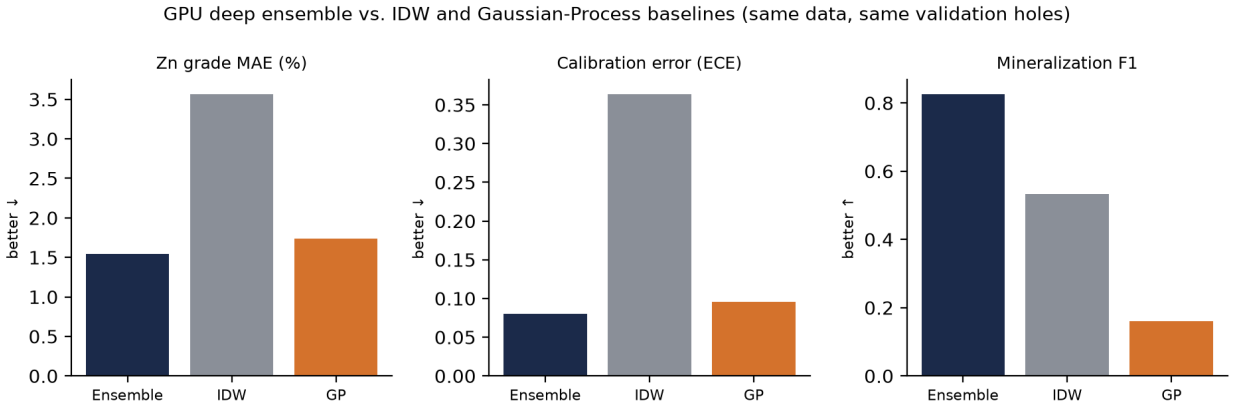



Figure 7. GPU ensemble vs. classical geostatistics baselines, identical data.

| Method | Zn MAE (%) | Zn RMSE (%) | ECE | F1 | Fit+predict time |
|---|---|---|---|---|---|
| GPU Deep Ensemble | 1.54 | 3.11 | 0.080 | 0.826 | 0.12 s |
| IDW (GPU, k=16) | 3.57 | 5.34 | 0.364 | 0.532 | 0.08 s |
| Gaussian Process | 1.74 | 2.69 | 0.096 | 0.160 | 22.78 s |

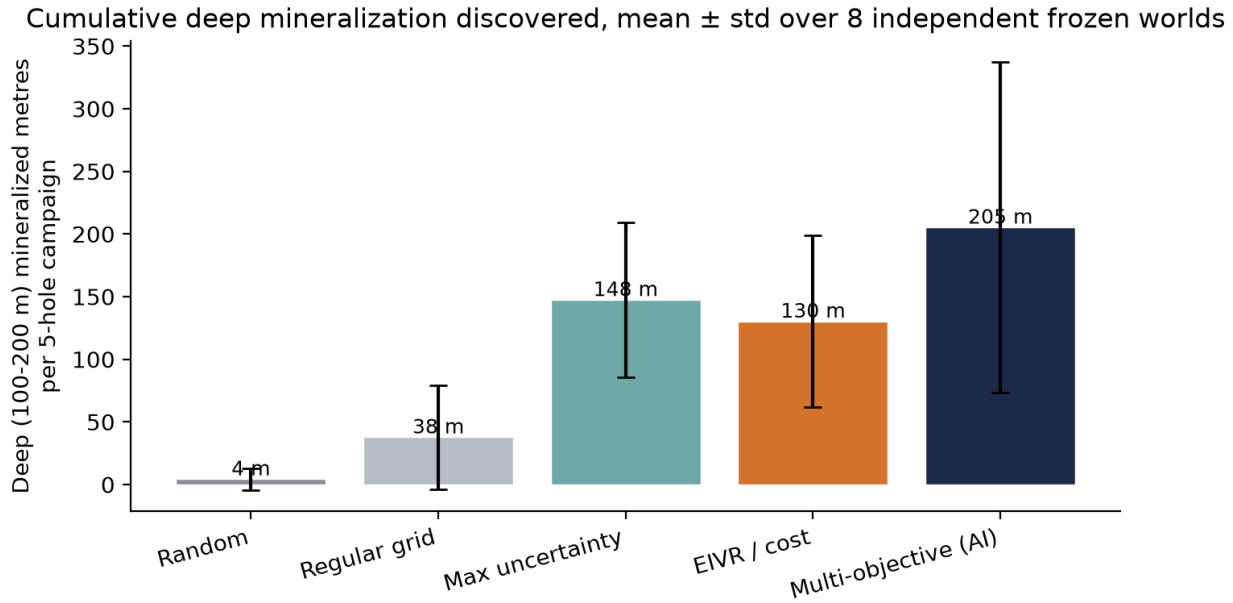



Figure 8. Average metres of deep ore found per 5-hole campaign, 200 total drilling decisions across 8 worlds.

| Strategy | Mean metres found | Std. dev. | Metres per $1,000 spent |
|---|---|---|---|
| Multi-objective (AI) | 205.3 | 131.7 | 0.536 |
| Max uncertainty | 147.5 | 61.8 | 0.383 |
| EIVR / cost | 130.3 | 68.8 | 0.339 |
| Regular grid | 37.8 | 41.6 | 0.108 |
| Random | 4.4 | 8.5 | 0.011 |

Note: this benchmark predates the Section 1 fix and used the earlier, unstable belief engine for its “multi-objective” strategy's hypothesis-discrimination term (a minority weight in that strategy's score). The core finding — adaptive strategies clearly beat random/grid baselines — does not depend on that term's precision and is not expected to change; re-running it with the fixed engine is listed in Section 6 as a next step.

# 6. What's Left to Improve

## 6.1 Already done as a direct result of this and the prior round of feedback

- **Fixed the belief-engine instability** (Section 1) — random sampling replaced with exact grid evaluation; verified deterministic (std = 0.000 across 10 reruns) where it was previously unstable (std ≈ 0.10-0.12).
- **Fixed the world-generator bug** that silently correlated independent worlds' target positions (discovered while building the cross-world test) and added a permanent regression test for it.
- **Replaced hardcoded drill-hole choices with real acquisition-driven selection** when regenerating this report's data, after discovering the previous script silently relied on stale, pre-fix hole choices that no longer matched the (regenerated) world.

## 6.2 Next

- Re-run the Section 5 benchmark with the fixed belief engine for full end-to-end consistency.
- Widen the Section 4.1 test beyond 8 worlds now that the generator bug is fixed, for a tighter estimate of the true accuracy spread.
- Investigate the overconfidence case from Figure 5 directly — a conformal-prediction wrapper around the ensemble would give a mathematical coverage guarantee instead of a hopeful estimate.
- Replace the statistical fault-plane fit with real structural wireframes once real mapping exists.
- Before any real Angouran data: run the whole pipeline on a public, real drill-hole dataset to pressure-test these findings against real (not synthetic) heterogeneity.
- When real data arrives: a retrospective back-test against a deposit's already-known deep extension is the single most important validation step, ahead of everything else on this list.

# Glossary

| Term | Meaning |
|---|---|
| ROC-AUC | 0-to-1 score for ranking “likely mineralized” above “likely barren.” 1.0 = perfect, 0.5 = coin flip, below 0.5 = backwards. |
| Effective sample size | Out of N random particles, how many are actually contributing meaningfully to the final answer. Can be far smaller than N if most particles are poor matches. |
| Importance-sampling degeneracy | The specific failure where a sharply-peaked true answer, combined with wide random guessing, means almost all guesses are wasted and a few lucky ones dominate — unstable results that don't improve much with more guesses. |
| Grid quadrature | Checking every evenly-spaced combination of a small number of unknowns directly, instead of guessing randomly. Zero randomness, so it gives the identical answer every time. |
| Zero-shot | Using a trained model on new data with no further training/adjustment. |
| Calibration / ECE | Whether a stated confidence is actually right that often, when checked against many predictions. 0 is perfect. |

Every number and figure in this report comes from real, reproducible program runs captured in technical_meta.json and crossworld2_result.json, cross-checked against the full 31-test automated suite (passing). The belief-engine fix is a real code change (src/optimization/hypotheses.py); the world-generator fix is covered by tests/test_geology.py::test_world_centres_are_well_spread.